

Linux_Wk12_S1 Lecture #29

System Monitoring & Performance Tuning

TABLE OF CONTENTS

Module 1: System Monitoring Tools

- Architecture Overview: How Monitoring Tools Work
- 1.1 top – The Classic Process Viewer
- 1.2 htop – Interactive Process Viewer
- 1.3 iostat – I/O Statistics
- 1.4 vmstat – Virtual Memory Statistics

Module 2: Performance Tuning

- Architecture: Where Tuning Happens
- 2.1 sysctl – Kernel Parameter Tuning
- 2.2 tuned – Dynamic System Tuning

Module 3: Log Management and Analysis

- 3.1 rsyslog – System Logging
- 3.2 journalctl – Systemd Journal Viewer

Module 4: Practical Labs

- Lab 1: Monitoring System Resources
- Lab 2: Detecting Memory Pressure
- Lab 3: I/O Performance Analysis
- Lab 4: Tuning for Desktop Performance
- Lab 5: Log Analysis and Troubleshooting

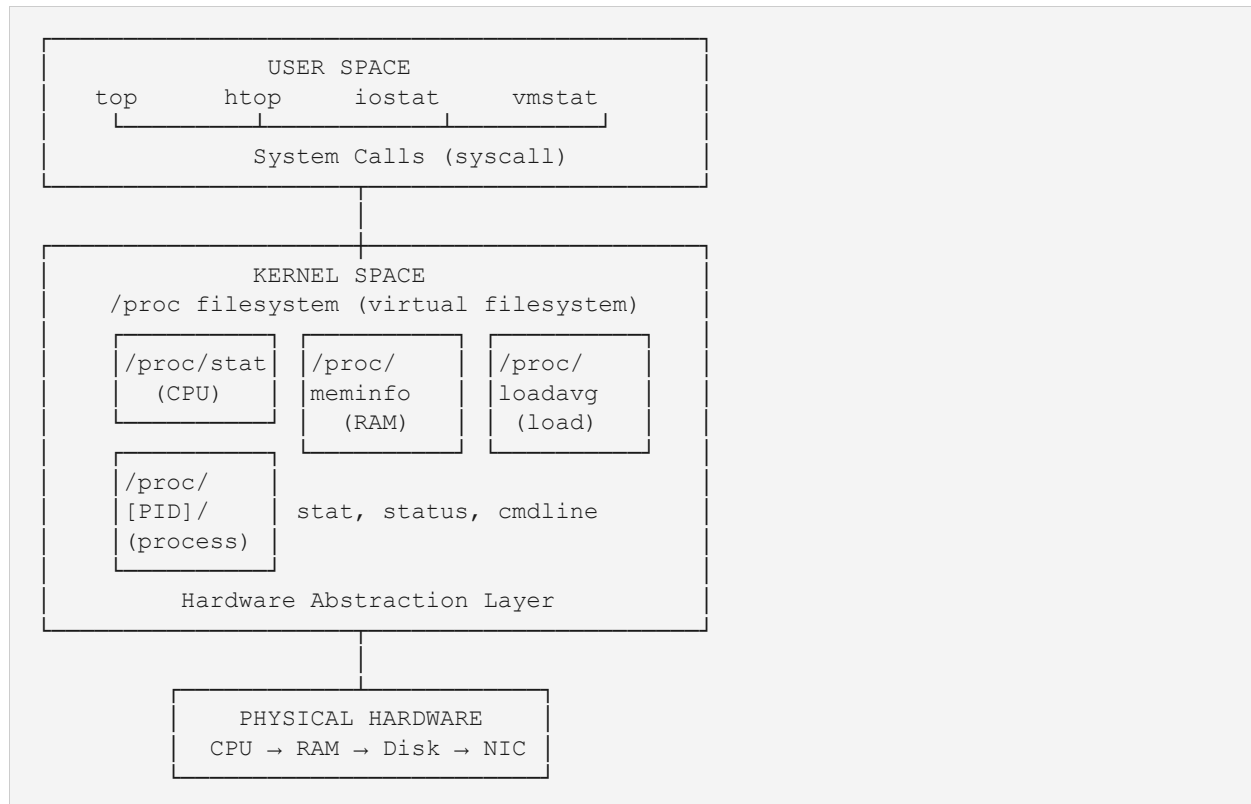
Additional Sections

- Troubleshooting & Common Pitfalls
- Quick Reference Card
- Keyboard Shortcuts (top & htop)
- Safety Reminders

Module 1: System Monitoring Tools

Architecture Overview: How Monitoring Tools Work

Before diving into commands, understand how these tools gather information. Tools like `top` read from the `/proc` directory — a virtual filesystem that the kernel updates in real-time. This is how they show live statistics without modifying the kernel.



1.1 top – The Classic Process Viewer

BACKGROUND & HISTORY

top was created in 1984 for BSD Unix and has been included in virtually every Linux distribution since 1991.

It remains the universal standard for process monitoring because it is available even on minimal systems

where newer tools are not installed.

top is like a 'dashboard' for your computer. Imagine driving a car: you have a speedometer, fuel gauge, and temperature gauge. top provides similar real-time gauges for your computer's CPU, memory, and running programs.

Key Concepts

Process

A running program (like Firefox, Terminal, or a background service). Each gets a unique PID (Process ID).

Process States

State	Name	Description
R	Running	Currently using CPU
S	Sleeping	Waiting for something (user input, disk, network)
T	Stopped	Paused by user or debugger
Z	Zombie	Finished but not cleaned up properly by parent

Load Average

ANALOGY

Think of a coffee shop with one cashier. Load average tells you how many customers are waiting in line.

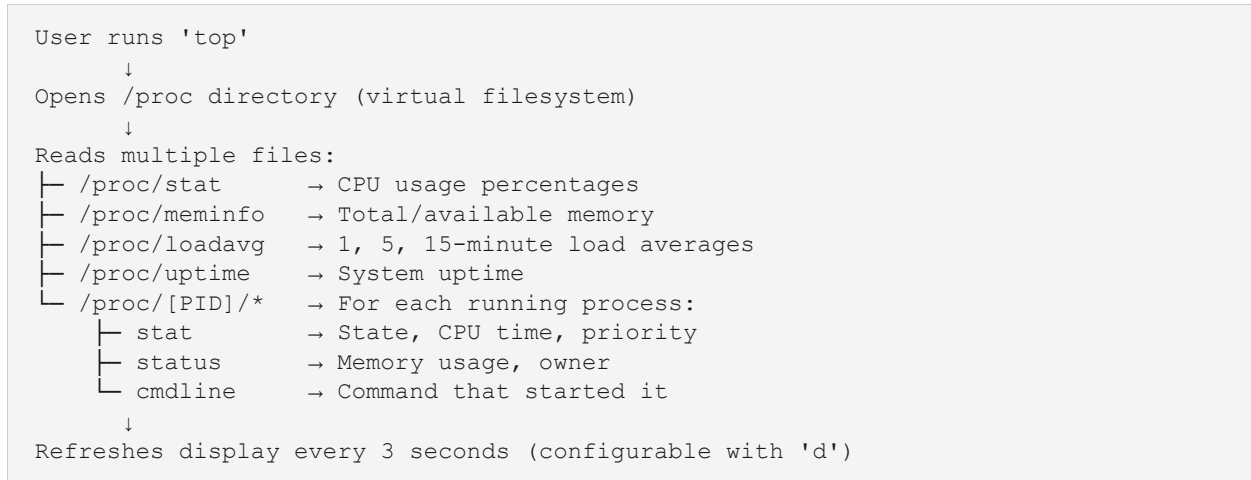
0.0 – 1.0 : No queue, cashier idle sometimes (good)

1.0 : Cashier busy but no waiting (optimal)

> 1.0 : Customers waiting in line (slow service)

On a 4-core CPU: 4.0 = fully utilised, > 4.0 = overloaded

How top Works (Architecture)



Basic Command

```
top
```

Interactive Commands (while top is running)

Key	Action	When to Use
h or ?	Help screen	Forgot a command
q	Quit	Done monitoring
M	Sort by memory usage	Finding memory hogs
P	Sort by CPU usage (default)	Finding CPU-intensive apps
T	Sort by total CPU time	Finding longest-running processes
k	Kill a process	Enter PID, then signal (9=force, 15=graceful)
r	Renice (change priority)	Lower priority of background tasks
1	Toggle CPU cores view	See individual core usage
c	Toggle command line display	See full path vs just name
V	Forest view (process tree)	See parent-child relationships
Space	Force refresh	Update display immediately
d	Change delay (refresh rate)	Make it update faster/slower

What Could Go Wrong?

Killing wrong process: Pressing k and entering the wrong PID can close important work. Always double-check the COMMAND column.

Killing system processes: Ending PID 1 (systemd) or low-numbered PIDs (< 100) can crash your system.

High refresh rate: Setting delay to 0.1 seconds uses more CPU just to run top itself.

Sample Output Explained

```
top - 14:32:10 up 2 days, 5:23,  2 users,  load average: 0.52, 0.58, 0.59
Tasks: 287 total,   1 running, 286 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.2 us,   2.1 sy,   0.0 ni, 92.1 id,   0.3 wa,   0.0 hi,   0.3 si,   0.0 st
MiB Mem :   7934.4 total,   2156.2 free,   3123.4 used,   2654.8 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,         0.0 used.  4456.2 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1842	student	20	0	4123456	156234	89123	S	8.5	1.9	2:34.56	firefox
2156	student	20	0	1234567	45678	23456	S	3.2	0.6	0:12.34	gnome-shell

Line-by-Line Breakdown

Line	What It Shows	Details
14:32:10 up 2 days...	System time & uptime	Current time, time since last boot (2 days, 5h 23m), users logged in
load average: 0.52, 0.58, 0.59	Average runnable processes	1-min, 5-min, 15-min averages. Healthy if < number of CPU cores
Tasks: 287 total...	Process counts	287 total: 1 running, 286 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.2 us...	CPU usage breakdown	User: 5.2%, System: 2.1%, Nice: 0%, Idle: 92.1%, I/O Wait: 0.3%
MiB Mem: 7934.4 total...	Physical memory	Total: 7.9 GB, Free: 2.1 GB, Used: 3.1 GB, Buffer/Cache: 2.6 GB
MiB Swap: 2048.0 total...	Virtual memory (swap)	Total: 2 GB, Free: 2 GB, Used: 0 GB — zero used is healthy

Process Columns Explained

Column	Full Name	What It Means	Good / Warning / Critical
PID	Process ID	Unique number identifying the process	N/A — just an identifier
USER	Owner	Which user started this process	Should match expected user
PR	Priority	Kernel scheduling priority (20 = normal)	0–20 normal; < 0 = real-time
NI	Nice Value	User-set priority (–20 to 19, negative = higher)	0 default; 10+ = low priority background
VIRT	Virtual Memory	Total memory allocated (including shared, swapped)	High is OK if RES is reasonable
RES	Resident Memory	Actual RAM currently in use — this matters most!	< 10% per app good; > 50% investigate
SHR	Shared Memory	Memory shared with other processes (usually libraries)	Usually small — informational
S	State	R=Running, S=Sleeping, T=Stopped, Z=Zombie	Zombie (Z) count should always be 0
%CPU	CPU Percentage	One full core = 100%; 4 cores max = 400%	Consistently > 100% per process → investigate
%MEM	Memory Percentage	Percent of total physical RAM used	< 5% normal; 5–20% heavy; > 50% possible leak
TIME+	CPU Time	Total CPU time this process has consumed	High = long-running or CPU-intensive process
COMMAND	Process Name	What program this process is running	Investigate unknown or unexpected names

Performance Indicators

Metric	Healthy	Warning	Critical
Load Average	< number of CPU cores	1.5× – 2× cores	> 2× cores (system overloaded)
%CPU (single process)	< 50%	50–90% sustained	100% sustained (may be stuck)
%MEM (single process)	< 10%	10–30%	> 50% (possible memory leak)
Zombie processes	0	1–2 temporary	Persistent zombies (parent process bug)
I/O Wait (wa)	< 5%	5–15%	> 20% (disk bottleneck)

Common Errors & Fixes

Error Message	Meaning	Solution
top: failed /proc/stat open	Permission denied or /proc not mounted	Run without sudo first; if persists, system corruption — reboot
top: unknown terminal type	TERM environment variable missing	Run: export TERM=xterm then retry top
top: cannot read /proc	Kernel procfs not available	Serious system error — reboot required
Process shows 0.0% CPU but won't die	Zombie or uninterruptible sleep	Check S column; if Z, restart the parent process
k command: 'Operation not permitted'	Trying to kill root or kernel thread	Do not kill PIDs < 100 (system processes)

1.2 htop – Interactive Process Viewer

BACKGROUND & HISTORY

htop was created in 2004 by Hisham Muhammad as an enhanced alternative to top. It was designed to be more user-friendly with colour coding, mouse support, and visual meters. Unlike top, it is NOT installed by default on most systems.

If top is a car's dashboard, htop is a modern digital cockpit. It shows the same information but with colours, graphs, and interactive elements that make it easier to understand at a glance.

Key Improvements over top

- Visual CPU/Memory bars instead of raw numbers
- Mouse support (click to select, scroll to navigate)
- Tree view to see process parent-child relationships
- Colour coding for different process types
- Easier process killing — F9 shows a signal selection menu

Installation

```
sudo apt update
sudo apt install htop
```

WARNING

Repository not found: If apt fails, check internet connection in VirtualBox:
Devices → Network → Enable Network Adapter
Disk full: If installation fails, check space with: df -h
VMs often have small root partitions (default 20 GB).

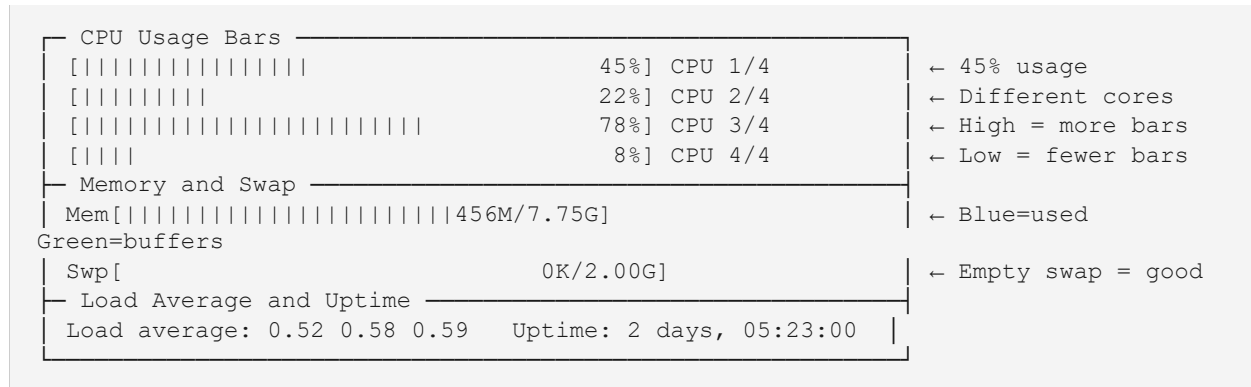
Launch

```
htop
```

Navigation

Input	Action	Details
Arrow Keys	Navigate process list	Up/Down to select; Left/Right for columns
Mouse Click	Select process	Click any line to select it
F1 or h	Help	Shows all commands and colour key
F2 or S	Setup	Change colours, meters, display options
F3 or /	Search	Find process by name (type to filter)
F4 or \	Filter	Show only matching processes
F5 or t	Tree View	Show parent-child process relationships
F6 or >	Sort	Choose column to sort by
F9 or k	Kill	Select signal: 15 = TERM (graceful), 9 = KILL (force)
F10 or q	Quit	Exit htop

Visual Elements Explained



Colour Coding — Process List

Colour	Meaning	What It Represents
Green	Low priority / User processes	Normal applications you started
Blue	Normal priority / System processes	OS services and daemons
Red	Kernel processes	Core OS functions — do NOT kill these!
Yellow	IRQ time	Hardware interrupt handling
Magenta	Soft IRQ	Network and disk interrupt handling
Cyan	Virtualisation	Time spent in VM hypervisor (VirtualBox overhead)

CPU Bar Colours (left to right within each bar)

Colour	Meaning
Blue	Low-priority (nice) processes
Green	Normal user processes
Red	Kernel / system processes
Yellow	IRQ (hardware interrupts)

Memory Bar Colours

Colour	Meaning
Green	Used by applications
Blue	Buffers (disk write cache)
Orange / Yellow	Cache (recently read files — can be reclaimed)

Common Errors & Fixes

Error / Problem	Solution
command 'htop' not found	sudo apt install htop
Error opening terminal: xterm-256color	Run: export TERM=xterm then retry
Cannot open /proc	Do not run htop with sudo unless necessary
Display garbled or corrupted	Resize terminal window larger, or run: reset
Colours look wrong	Press F2 → Colors → select a different scheme

1.3 iostat – I/O Statistics

BACKGROUND & HISTORY

iostat is part of the sysstat package, first released in 1998.

It was originally written by Sebastien Godard to provide detailed I/O statistics that vmstat couldn't show.

It is the standard tool for diagnosing disk performance issues.

iostat is like a speedometer for your hard drive. While top shows CPU and memory, iostat shows how fast data is being read/written and whether your disk is the performance bottleneck.

VIRTUALBOX NOTE

Disk I/O metrics inside a VM reflect virtual disks (.vdi files), not your physical hard drive/SSD.

The performance you see is limited by:

- Host disk speed (if host is busy, VM is slow)
- VirtualBox disk controller type (SATA vs NVMe vs VirtIO)
- Whether 'Host I/O Cache' is enabled in VM settings

Key Concepts

Term	Definition
IOPS	Input/Output Operations Per Second — how many requests the disk handles
Throughput	MB/s — how much data is actually transferred per second
Queue Depth (aqu-sz)	Number of I/O requests waiting in line to be processed
Await	Average time in milliseconds to complete one I/O request (lower is better)

□ ANALOGY

I/O Wait (%wa) is like a cashier waiting for the coffee machine to finish brewing.

The cashier (CPU) is ready to work but cannot do anything until the machine (disk) responds.

High %wa = CPU is idle because it is waiting for a slow disk.

Installation

```
sudo apt install sysstat
```

```
# Enable and start the data collection service
```

```
sudo systemctl enable sysstat
```

```
sudo systemctl start sysstat
```

Practical Usage

Basic command

```
iostat
```

Extended statistics with human-readable sizes

```
iostat -h -x 1 5
# -h : Human-readable (KB, MB, GB)
# -x : Extended statistics (more columns)
# 1 : Update every 1 second
# 5 : Show 5 iterations then stop
```

Sample Output Explained

```
Linux 6.8.0-40-generic (ubuntu-vm) 04/10/2026 _x86_64_ (4 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           8.25    0.01    3.45    1.23    0.00   87.06

Device    r/s    w/s  rkB/s  wkB/s  rrqm/s  wrqm/s  r_await  w_await  aqu-sz   %util
sda       2.34   15.67 125.40 456.78   0.12    8.90    3.45    5.67    0.09    0.81
```

CPU Section (avg-cpu)

Column	Meaning	Healthy	Warning	Critical
%user	User applications CPU	0–70%	70–90%	> 90% sustained
%nice	Low-priority user CPU	Varies	—	—
%system	Kernel/OS CPU	0–20%	20–40%	> 40% (investigate)
%iowait	Waiting for disk	0–5%	5–15%	> 20% (disk bottleneck)
%steal	VM hypervisor time	0–1%	1–5%	> 5% (host overloaded)
%idle	CPU doing nothing	10–100%	0–10%	0% (maxed out)

VIRTUALBOX NOTE

%steal shows time VirtualBox took your CPU to give to other VMs or the host.
If steal is high, close other VMs or give this VM more CPU cores in VirtualBox settings.

Device Section (Disk Stats)

Column	Full Name	Meaning	Target
r/s	Reads per second	I/O read operations per second	Varies by workload
w/s	Writes per second	I/O write operations per second	Varies by workload
rkB/s	Read KB/s	Data read throughput	SSD > 100 MB/s; HDD > 50 MB/s
wkB/s	Write KB/s	Data write throughput	SSD > 100 MB/s; HDD > 50 MB/s
rrqm/s	Read requests merged	Requests combined for efficiency	Higher = better efficiency
wrqm/s	Write requests merged	Requests combined for efficiency	Higher = better efficiency
r_await	Read await time	ms to complete one read request	< 10 ms ideal
w_await	Write await time	ms to complete one write request	< 10 ms ideal
aqu-sz	Average queue size	Requests waiting to be processed	< 1 (> 5 = backing up)
svctm	Service time	OBSOLETE — ignore this column	N/A
%util	Device utilisation	% of time the disk was busy	< 70% good; > 80–90% = saturated

Critical Indicators

Indicator	Healthy	Warning	Critical	Action
%util	0–60%	60–80%	> 80–90% sustained	Add faster disk or reduce I/O
await	0–10 ms	10–20 ms	> 20 ms	Slow disk or too many requests
aqu-sz	0–1	1–5	> 5	Disk cannot keep up with requests
%iowait	0–5%	5–15%	> 20%	CPU starved waiting for disk

Practical Exercise – Detecting Disk Bottleneck

```
# Terminal 1: Monitor disk I/O continuously
iostat -x 1 30

# Terminal 2: Create heavy disk activity (writes 1 GB of zeros)
dd if=/dev/zero of=/tmp/stress_test.bin bs=1M count=1024

# Watch in Terminal 1:
#   %util should spike to near 100%
#   w/s and kB/s will jump up significantly
#   aqu-sz may increase (queue building up)
#   await will increase (possibly > 20 ms)

# In VirtualBox you may see lower throughput – normal due to virtualisation
overhead

# Clean up when done
rm /tmp/stress_test.bin
```

Common Errors & Fixes

Error / Problem	Solution
iostat: command not found	sudo apt install sysstat
Cannot open /proc/diskstats	Run with sudo or check that /proc is mounted
No data available	sudo systemctl enable --now sysstat
All zeros in output	Normal in some VMs; generate I/O with the dd exercise above

1.4 vmstat – Virtual Memory Statistics

BACKGROUND & HISTORY

vmstat (Virtual Memory Statistics) was originally written by Bill Joy and Coby Raber for BSD Unix in the late 1970s.

It was designed to help administrators understand virtual memory pressure.

It is still essential today because it shows system-wide behaviour that process-specific tools miss.

While top shows what individual processes are doing, vmstat shows what the entire system is doing — memory pressure, swapping, I/O patterns, and CPU scheduling. It is like checking the health of the whole engine, not just individual cylinders.

Key Concepts

Concept	Definition
Swapping	Moving entire memory pages to/from disk — very slow, indicates RAM shortage
Paging	Moving memory between RAM and disk — normal in small amounts
Context Switches	CPU switching between processes — high count = busy system
Interrupts	Hardware signals (keyboard, disk, network) requiring CPU attention

ANALOGY

Swapping is like moving files from your desk to a filing cabinet down the hall because your desk is too small.

It works, but everything slows down because you keep walking back and forth.

si (swap in) and so (swap out) both equal 0 is the ideal — means no swapping is happening.

Practical Usage

```

vmstat                # One-shot output
vmstat 1 10           # 1-second intervals, 10 samples
vmstat -w -a 1 5      # Wide output, active/inactive memory, 5 samples
# -w : Wide output (easier to read)
# -a : Show active/inactive memory breakdown
    
```

Sample Output Explained

```

procs  -----memory-----  ---swap--  -----io-----  -system--  -----cpu-----
r  b    swpd   free   buff  cache   si   so    bi    bo    in   cs  us  sy  id  wa  st
1  0      0 2205632 145678 2345678    0    0    12    45   234  567  5   2  93  0   0
0  0      0 2201234 145680 2345680    0    0     0     0  123  234  3   1  96  0   0
1  0      0 2198765 145682 2345690    0    0     4    12  456  789  8   3  89  0   0
    
```


PROCESSES Section (procs) — Run Queue

Column	Meaning	Healthy	Warning	Critical
r	Runnable — waiting for CPU	0 – (# of cores)	1.5× – 2× cores	> 2× cores = system overloaded
b	Blocked — waiting for I/O	0	1–3	> 3 (I/O bottleneck)

MEMORY Section

Column	Meaning	Healthy	Warning Signs
swpd	Virtual memory used (KB)	0	> 0 = swap is active — BAD
free	Idle memory (KB)	> 10% of total	< 5% of total RAM
buff	Buffer cache (KB)	Varies	—
cache	Page cache (KB)	Varies — large cache is good	—
si	Swap IN (KB/s) — reading from swap	0	> 0 = system is RAM-starved
so	Swap OUT (KB/s) — writing to swap	0	> 0 = system is RAM-starved

I/O Section

Column	Meaning	Typical Values
bi	Blocks IN from disk (blocks/s)	0–1000 (depends on workload)
bo	Blocks OUT to disk (blocks/s)	0–1000 (depends on workload)

SYSTEM Section

Column	Meaning	Healthy	Warning
in	Interrupts per second	100–1000	> 10,000 (possible IRQ storm)
cs	Context switches per second	1000–10,000	> 50,000 (excessive switching)

CPU Section

Column	Meaning	Healthy	Warning	Critical
us	User time (apps)	0–70%	70–90%	> 90%
sy	System time (kernel)	0–20%	20–40%	> 40%
id	Idle time	> 20%	5–20%	< 5% (fully maxed)
wa	I/O wait time	< 5%	5–15%	> 20% (disk bottleneck)
st	Steal time (VM hypervisor)	0%	1–5%	> 5% (host overloaded)

Performance Indicators Summary

Indicator	Healthy	Warning	Critical	Action Required
r (run queue)	< # of CPU cores	1.5× – 2× cores	> 2× cores	Add CPU or reduce processes
b (blocked)	0	1–3	> 3	Check disk I/O with iostat
si/so (swap)	0 / 0	Occasional spikes	Continuous > 0	Add RAM or reduce swappiness
wa (I/O wait)	< 5%	5–15%	> 20%	Upgrade disk or reduce I/O
id (idle)	> 20%	5–20%	< 5%	CPU fully utilised
st (steal)	0%	1–3%	> 5%	Host overloaded — reduce VM count

Common Errors & Fixes

Error / Problem	Solution
vmstat: command not found	sudo apt install procps
cannot open /proc/meminfo	Run without sudo; check: mount grep proc
All values zero or static	Run vmstat 1 (continuous) to see live updates
Negative numbers in output	Rare kernel counter overflow — usually harmless; reboot if concerned

Module 2: Performance Tuning

Architecture: Where Tuning Happens

TUNING LAYERS
1. APPLICATIONS (Firefox, VS Code, etc.) └ Config files in ~/.config/
2. SYSTEM SERVICES (systemd) └ systemctl edit <service>
3. KERNEL PARAMETERS (sysctl) └ /proc/sys/ → /etc/sysctl.conf └ VM, network, filesystem behaviour
4. DYNAMIC TUNING DAEMONS (tuned) └ Monitors and adjusts automatically └ Profiles for different workloads
5. HARDWARE/BIOS (outside VM) └ CPU governors, power states └ Limited in VirtualBox (host controls this)

2.1 sysctl – Kernel Parameter Tuning

BACKGROUND & HISTORY

sysctl was introduced in 4.4 BSD (1994) to provide a safe interface for kernel tuning. Before this, administrators had to write directly to /proc files — risky and error-prone. The Linux version was adapted by Andries Brouwer in 1999. It remains the standard for viewing and modifying kernel behaviour without recompiling.

The Linux kernel has hundreds of adjustable parameters controlling memory management, networking, filesystems, and security. sysctl is the safe, official way to view and change these parameters.

ANALOGY

Kernel parameters are like the settings menu on your phone. sysctl is the settings app that lets you safely change brightness, volume, etc. You could also open the phone and adjust wires directly (writing to /proc), but that is dangerous.

CRITICAL WARNING

Changing kernel parameters incorrectly can make your system unstable or UNBOOTABLE. Always:

1. Test changes temporarily first — they reset after reboot
2. Document original values before changing anything
3. Take a VM snapshot before applying changes
4. Never change more than one parameter at a time

Viewing Parameters

```
sysctl -a | less           # List ALL parameters (piped to pager)
sysctl -a | grep vm.swappiness # Search for specific parameter
sysctl -a | grep net.ipv4    # Search network parameters
sysctl vm.swappiness         # Read one specific parameter
# Output: vm.swappiness = 60
```

Common Tunable Parameters

1. Virtual Memory (VM) Settings

```
# View current swappiness value
sysctl vm.swappiness

# Swappiness range: 0 to 100
# 0   = never swap unless absolutely necessary
# 60  = default (Ubuntu desktop)
# 100 = swap aggressively
```

ANALOGY

vm.swappiness is like deciding how quickly to move papers from your desk drawer to the filing cabinet.

Low value (10): Keep papers on the desk (RAM) as long as possible, even if crowded.

High value (60–100): Quickly move papers to the cabinet (swap) to keep the desk clear.

2. Network Settings

```
# Enable IP forwarding (required for NAT, routing)
sudo sysctl net.ipv4.ip_forward=1

# SYN flood protection (DDoS mitigation)
sudo sysctl net.ipv4.tcp_syncookies=1

# Increase connection tracking table size
sudo sysctl net.netfilter.nf_conntrack_max=65536
```

3. Kernel Scheduler

```
# Process scheduling granularity (nanoseconds)
sysctl kernel.sched_min_granularity_ns

# Latency target for scheduling decisions
sysctl kernel.sched_latency_ns
```

Applying Changes

Temporary change (resets on reboot)

```
# BEFORE: Check current value
sysctl vm.swappiness
# Output: vm.swappiness = 60

# Apply change
sudo sysctl vm.swappiness=10
# Output: vm.swappiness = 10

# AFTER: Verify
sysctl vm.swappiness
# Output: vm.swappiness = 10

# Note: This resets to 60 after reboot unless made permanent
```

Making Changes Permanent — Method 1: Custom config file (RECOMMENDED)

```
# Create a new sysctl configuration file
sudo nano /etc/sysctl.d/99-custom.conf
```

```
# Content of the file:
# Memory settings for desktop VM
vm.swappiness=10
vm.dirty_ratio=15
vm.dirty_background_ratio=5
vm.vfs_cache_pressure=50

# Network settings
net.ipv4.tcp_syncookies=1
net.ipv4.ip_forward=1
net.core.somaxconn=65535

# Security settings
kernel.dmesg_restrict=1
```

```
# Apply all config files from /etc/sysctl.d/
sudo sysctl --system
```

Making Changes Permanent — Method 2: Edit main config (less recommended)

```
sudo nano /etc/sysctl.conf
```

Practical Exercise: Optimising for Desktop VM Use

```
# === STEP 1: Check current settings (BEFORE) ===
echo '=== CURRENT SETTINGS (Before) ==='
sysctl vm.swappiness
sysctl vm.vfs_cache_pressure
sysctl vm.dirty_ratio
sysctl vm.dirty_background_ratio

# === STEP 2: Apply desktop optimisations (temporary for testing) ===
echo '=== APPLYING CHANGES ==='
sudo sysctl vm.swappiness=10
sudo sysctl vm.vfs_cache_pressure=50
sudo sysctl vm.dirty_ratio=10
sudo sysctl vm.dirty_background_ratio=5

# === STEP 3: Verify changes (AFTER) ===
echo '=== NEW SETTINGS (After) ==='
sysctl vm.swappiness
sysctl vm.vfs_cache_pressure

# Test for several hours/days...

# === STEP 4: If satisfied, make permanent ===
echo '=== MAKING PERMANENT ==='
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-desktop-
tweaks.conf
echo 'vm.vfs_cache_pressure=50' | sudo tee -a /etc/sysctl.d/99-desktop-
tweaks.conf
echo 'vm.dirty_ratio=10' | sudo tee -a /etc/sysctl.d/99-desktop-
tweaks.conf
echo 'vm.dirty_background_ratio=5' | sudo tee -a /etc/sysctl.d/99-desktop-
tweaks.conf

# Apply
sudo sysctl --system
```

Parameter Reference

Parameter	Default	Recommended for VM	Effect
vm.swappiness	60	10–30	Lower = prefer RAM over swap
vm.vfs_cache_pressure	100	50	Lower = keep directory/inode cache longer
vm.dirty_ratio	20	10–15	Max % memory before forced disk write
vm.dirty_background_ratio	10	5	% memory to start background writeback
net.core.somaxconn	4096	65535	Max connection backlog queue

Before / After Comparison

Parameter	Before (Default)	After	Observed Effect
vm.swappiness	60	10	Less swapping; more responsive desktop
vm.vfs_cache_pressure	100	50	Filesystem cache kept longer; faster repeated file access
vm.dirty_ratio	20	10	Fewer large write bursts to disk
vm.dirty_background_ratio	10	5	Earlier, smoother background disk writes

WARNING

What Could Go Wrong?

vm.swappiness too low (0–5): System may kill processes (OOM killer) instead of swapping when RAM is full.

vm.dirty_ratio too low: Excessive and frequent disk writes, reducing performance.

Wrong syntax in config file: System may fail to boot. Always validate with: `sudo sysctl --system`

Common Errors & Fixes

Error Message	Meaning	Solution
sysctl: cannot stat /proc/sys/...: No such file	Parameter doesn't exist or module not loaded	Check spelling; kernel module may need loading
sysctl: permission denied	Trying to modify without sudo	Use sudo for all write operations
sysctl: setting key: Invalid argument	Wrong value type (e.g. string instead of number)	Ensure value is numeric and in valid range
Settings not persistent after reboot	Config file not created	Add to /etc/sysctl.d/*.conf and run <code>sudo sysctl --system</code>
System won't boot after changes	Invalid parameters in boot config	Boot recovery mode; remove bad config from /etc/sysctl.d/

2.2 tuned – Dynamic System Tuning

BACKGROUND & HISTORY

tuned was developed by Red Hat in 2009 for RHEL 6 to solve a common problem: static tuning works for one workload but hurts others. It uses monitoring daemons to adjust settings dynamically. Ported to Ubuntu in 2018 — now the recommended way to manage performance profiles.

Theory

While sysctl sets static values, tuned is like having an autopilot. It monitors your system and switches between 'profiles' optimised for different scenarios: desktop use, server workloads, virtual machines, power saving, etc.

How tuned Works

1. Daemon (tuned) runs in the background continuously
2. Reads the active profile configuration file
3. Monitors CPU, disk, and network usage patterns
4. Adjusts sysctl parameters, CPU governors, disk settings dynamically

Installation

```
sudo apt install tuned tuned-utils tuned-utils-systemtap
```

WARNING

If package is not found, enable the universe repository first:

```
sudo add-apt-repository universe  
sudo apt update
```

Managing the Service

```
sudo systemctl status tuned      # Check current status  
sudo systemctl start tuned       # Start the service  
sudo systemctl enable tuned      # Enable auto-start on boot  
sudo systemctl restart tuned     # Restart if needed
```

Available Profiles

```
sudo tuned-adm list
```

Available profiles:

- | | |
|---------------------------|---|
| - balanced | - General non-specialised tuned profile |
| - desktop | - Optimise for the desktop use-case |
| - hpc-compute | - Optimise for HPC compute workloads |
| - latency-performance | - Optimise for deterministic performance |
| - network-latency | - Optimise for low-latency networking |
| - network-throughput | - Optimise for streaming network throughput |
| - powersave | - Optimise for low power consumption |
| - throughput-performance | - Broadly applicable high-throughput tuning |
| - virtual-guest | - Optimise for running inside a virtual machine |
| - virtual-host | - Optimise for running KVM guests |
| - accelerator-performance | - Optimise for AI/ML accelerated workloads |

Current active profile: balanced

Applying Profiles

```
sudo tuned-adm active           # Check current profile
sudo tuned-adm profile desktop  # Apply desktop profile
sudo tuned-adm profile virtual-guest # Apply VM guest profile
(recommended for VirtualBox)
sudo tuned-adm profile throughput-performance # Apply throughput profile
sudo tuned-adm off              # Turn off tuned (manual control)
```

TIP

VirtualBox Recommendation:

For Ubuntu Desktop 24.04 in VirtualBox, use:

- virtual-guest — conservative, safe for VMs
- desktop — if VM has 4 GB+ RAM and 2+ cores
- balanced — default, a good starting point

Profile Details

Desktop Profile (/usr/lib/tuned/desktop/tuned.conf)

```
[main]
summary=Optimise for the desktop use-case
include=balanced

[sysctl]
# Less swapping for better desktop responsiveness
vm.swappiness=10

[cpu]
# Keep CPU in performance mode
governor=performance
```

Virtual Guest Profile (/usr/lib/tuned/virtual-guest/tuned.conf)

```
[main]
summary=Optimise for running inside a virtual guest
include=throughput-performance

[sysctl]
# Reduce disk flushing for better VM performance
vm.dirty_ratio=30
vm.dirty_background_ratio=10
vm.swappiness=30

[disk]
# Optimise for virtual disk controllers
readahead=>4096
```

Creating a Custom Profile

```
# Create custom profile directory
sudo mkdir -p /etc/tuned/student-vm

# Create configuration file
sudo nano /etc/tuned/student-vm/tuned.conf
```

```
[main]
summary=Optimised for student VMs in VirtualBox
include=virtual-guest

[sysctl]
# Conservative memory settings for 4-8 GB VMs
vm.swappiness=20
vm.dirty_ratio=20
vm.dirty_background_ratio=10
vm.vfs_cache_pressure=100

[vm]
# Transparent hugepages – can help or hurt depending on workload
transparent_hugepages=madvise

[cpu]
# Balance performance and power
governor=ondemand

[disk]
# Good readahead for development workloads
readahead=256
```

```
sudo tuned-adm profile student-vm
```

Verification Commands

```
sudo tuned-adm verify      # Verify tuned is applying settings correctly
sudo tuned-adm profile_info # Show detailed info about current profile
sudo tuned-adm active      # Show active profile name
sudo tuned-adm list        # List all available profiles
```

Common Errors & Fixes

Error / Problem	Solution
tuned-adm: command not found	sudo apt install tuned
No active profiles	sudo systemctl start tuned
Profile 'xyz' not found	Run sudo tuned-adm list to see exact correct profile names
Cannot obtain lock	sudo systemctl restart tuned
Changes do not apply	Check sudo tuned-adm verify; look for syntax errors in profile

Module 3: Log Management and Analysis

3.1 rsyslog – System Logging

BACKGROUND & HISTORY

rsyslog (reliable syslog) was created in 2004 by Rainer Gerhards as a replacement for traditional syslog. It added TCP transmission, encryption, and high-precision timestamps. Ubuntu has used it as the default logger since version 8.04 (2008). Currently maintained by the Adiscon company.

Linux systems generate thousands of events per minute: logins, program errors, hardware status, security alerts. Logging captures these in files for troubleshooting, security auditing, and performance analysis.

Log Severity Levels (0 = most severe, 7 = least severe)

Level	Name	Description	Example
0	emerg	System is unusable	Kernel panic
1	alert	Action must be taken immediately	Disk full, database corrupted
2	crit	Critical conditions	Hardware failure
3	err	Error conditions	Service failed to start
4	warning	Warning conditions	Deprecated feature used
5	notice	Normal but significant condition	New user account created
6	info	Informational messages	Service started successfully
7	debug	Debug-level messages	Detailed function call traces

Log File Locations

File	Contents	Typical Size
/var/log/syslog	General system activity	10–100 MB
/var/log/auth.log	Authentication attempts (login, sudo)	1–10 MB
/var/log/kern.log	Kernel messages	5–50 MB
/var/log/dmesg	Boot messages (hardware detection)	100 KB – 1 MB
/var/log/apt/history.log	Package management history	1–5 MB
/var/log/boot.log	Boot process messages	~100 KB
/var/log/cron.log	Cron job execution records	Varies
/var/log/mail.log	Mail server messages (if installed)	Varies
/var/log/apache2/	Web server logs (if installed)	Can be very large

Logs in VMs can grow quickly and fill small virtual disks (VirtualBox default is 20 GB).
 Always monitor /var/log size with: `du -sh /var/log`

Viewing Logs with Standard Tools

```

cat /var/log/syslog                # View entire log file
tail /var/log/syslog              # View last 10 lines
tail -f /var/log/syslog           # Follow new entries in real-time
tail -n 100 /var/log/syslog       # Last 100 lines

# Search logs
grep -i error /var/log/syslog     # Search for errors (case-insensitive)
grep sshd /var/log/auth.log       # Search for specific service
grep -C 3 'Failed password' /var/log/auth.log # 3 lines of context around match

# Filter by date
grep "$(date +%b\ %e)" /var/log/syslog # Today's entries
grep 'Apr 10' /var/log/syslog      # Specific date
    
```

Log Rotation

Logs are rotated automatically to prevent disk space exhaustion. Configuration lives in `/etc/logrotate.conf` and `/etc/logrotate.d/`

```
# View main logrotate configuration
cat /etc/logrotate.conf

# View rsyslog-specific rotation config
cat /etc/logrotate.d/rsyslog

# Force rotation immediately (for testing)
sudo logrotate -f /etc/logrotate.conf
```

Sample logrotate configuration

```
/var/log/syslog
{
    rotate 7          # Keep 7 rotated (archived) copies
    daily             # Rotate once per day
    missingok         # OK if file doesn't exist - don't error
    notifempty        # Don't rotate empty files
    compress          # Compress old logs with gzip
    delaycompress     # Don't compress the most recently rotated
    sharedscripts     # Run scripts once for all matching files
    postrotate
        /usr/lib/rsyslog/rsyslog-rotate > /dev/null 2>&1 || true
    endscript
}
```

Configuring rsyslog

Main configuration: `/etc/rsyslog.conf` Custom rules: `/etc/rsyslog.d/`

```
# View default rules
ls /etc/rsyslog.d/
cat /etc/rsyslog.d/50-default.conf
```

Understanding rsyslog Rules

```
# Format: selector    action
# selector: facility.priority
# facility: auth, cron, daemon, kern, mail, user, local0-7, etc.
# priority: debug, info, notice, warning, err, crit, alert, emerg

# Examples from 50-default.conf:
auth,authpriv.*          /var/log/auth.log
*.*;auth,authpriv.none   -/var/log/syslog
cron.*                   /var/log/cron.log
daemon.*                 -/var/log/daemon.log
kern.*                   -/var/log/kern.log
mail.*                   -/var/log/mail.log
user.*                   -/var/log/user.log
```

Creating a Custom Logging Rule

```
# Create a file to log all debug messages separately
sudo nano /etc/rsyslog.d/10-debug.conf

# Add this line to the file:
*.debug    /var/log/debug.log

# Restart rsyslog to apply
sudo systemctl restart rsyslog
```

Common Errors & Fixes

Error / Problem	Solution
Permission denied reading logs	Use sudo, or add user to adm group: <code>sudo usermod -a -G adm student</code> then logout/login
rsyslogd: unknown facility	Check valid facilities: auth, cron, daemon, kern, mail, user, local0-7
Logs not updating	<code>sudo systemctl restart rsyslog</code>
Disk full due to logs	<code>sudo logrotate -f /etc/logrotate.conf</code> or check: <code>df -h</code>
imjournal: journal files not readable	Check permissions on <code>/var/log/journal</code>

3.2 journalctl – Systemd Journal Viewer

BACKGROUND & HISTORY

journalctl was introduced with systemd in 2010 by Lennart Poettering. It replaced the fragmented logging of early Linux with a centralised, structured, indexed journal. Unlike text-based logs, it stores binary data with metadata, enabling powerful queries. Became default in Ubuntu with version 15.04 (2015).

Traditional logs are text files that programs append to. The systemd journal is a database of events with structured fields (PID, UID, service name, boot ID, etc.). This allows complex queries like: 'show me all errors from the SSH service during the last boot.'

Advantages over Traditional Syslog

Advantage	Details
Structured metadata	Every entry includes PID, UID, GID, unit name, and more
Indexed for fast search	Can search millions of log entries in seconds
Boot-aware	Logs are tagged with boot ID — easy to compare across reboots
Integrated with systemd	Service start/stop events captured automatically
Captures early boot	Stores messages that syslog misses (before rsyslog starts)

Basic Usage

```
journalctl          # View all logs (oldest first, opens in pager)
journalctl -n 100   # Last 100 lines
journalctl -f        # Follow new entries in real-time (like tail -f)
journalctl -r        # Reverse order – newest entries first
```

Filtering by Time

```
journalctl --since today
journalctl --since '2026-04-10 10:00:00'
journalctl --since '2026-04-10 09:00' --until '2026-04-10 12:00'
journalctl --since '1 hour ago'

# By boot:
journalctl -b        # Current boot only
journalctl -b -1     # Previous boot
journalctl -b -2     # Two boots ago
```

Filtering by Service (Unit)

```
journalctl -u ssh          # SSH service logs
journalctl -u ssh.service  # Same – .service extension optional
journalctl -u nginx.service # nginx web server logs
journalctl -u ssh -u nginx # Multiple services at once
journalctl -u ssh -f       # Follow SSH logs in real-time
```

Filtering by Priority

```
journalctl -p emerg..alert      # Emergency and alert only
journalctl -p err               # Errors and above
journalctl -p warning          # Warnings and above
journalctl -p info             # Info and above (very verbose)
journalctl -p debug            # All messages including debug (very verbose)
```

Filtering by Process, User, or Executable

```
journalctl _PID=1234           # By specific process ID
journalctl _UID=1000           # By user ID
journalctl _GID=1000           # By group ID
journalctl /usr/bin/bash       # By executable path
journalctl /usr/sbin/sshd      # SSH daemon specifically
```

Advanced Filtering

Kernel messages only

```
journalctl -k                  # Current boot kernel messages
journalctl -k -b -1            # Previous boot kernel messages
```

Listing all recorded boots

```
journalctl --list-boots

# Sample output:
-5 abc123... Mon 2026-04-06 08:00 Mon 2026-04-06 18:00
-4 def456... Tue 2026-04-07 08:00 Tue 2026-04-07 18:00
-3 ghi789... Wed 2026-04-08 08:00 Wed 2026-04-08 18:00
-2 jkl012... Thu 2026-04-09 08:00 Thu 2026-04-09 18:00
-1 mno345... Fri 2026-04-10 08:00 Fri 2026-04-10 18:00
 0 pqr678... Sat 2026-04-11 08:00 Sat 2026-04-11 11:00 ← current boot

journalctl -b -5                # View logs from 5 boots ago
```

JSON output (for scripting and automation)

```
journalctl -o json              # Full JSON (one object per line)
journalctl -o json-pretty       # Pretty-printed JSON (human-readable)
journalctl -o verbose           # Verbose output - all available fields shown
```

Log Maintenance

Check disk usage

```
journalctl --disk-usage
# Output: Archived and active journals take up 100.0M in the file system.
```

WARNING

Journal logs can consume gigabytes of disk space in busy systems.
In a 20 GB VirtualBox VM, this can fill your root partition and crash the system.
Always monitor journal size before it becomes a problem.

Cleaning up old logs

```
sudo journalctl --vacuum-size=100M    # Retain only last 100 MB
sudo journalctl --vacuum-time=7d      # Retain only last 7 days
sudo journalctl --vacuum-files=10     # Retain only last 10 boot files
```

Persistent configuration

```
sudo nano /etc/systemd/journald.conf

# Recommended settings for a VirtualBox VM:
[Journal]
Storage=persistent      # Keep logs across reboots
SystemMaxUse=500M       # Maximum total disk usage (500 MB for VMs)
SystemMaxFileSize=100M  # Maximum individual journal file size
MaxFileSec=1week        # Rotate files after 1 week

# Apply changes:
sudo systemctl restart systemd-journald
```

Sample Output Analysis

```
Apr 10 14:32:15 ubuntu-vm sshd[1234]: Failed password for invalid user admin from
192.168.1.100 port 54322 ssh2
Apr 10 14:32:18 ubuntu-vm sshd[1234]: Connection closed by authenticating user
admin 192.168.1.100 port 54322 [preauth]
Apr 10 14:32:20 ubuntu-vm sshd[1235]: Accepted password for student from
192.168.1.50 port 54324 ssh2
Apr 10 14:32:20 ubuntu-vm sshd[1235]: pam_unix(sshd:session): session opened for
user student(uid=1000) by (uid=0)
Apr 10 14:32:20 ubuntu-vm systemd[1]: Created slice User Slice of UID 1000.
Apr 10 14:32:20 ubuntu-vm systemd[1]: Starting User Runtime Directory
/run/user/1000...
Apr 10 14:32:20 ubuntu-vm systemd[1]: Finished User Runtime Directory
/run/user/1000.
Apr 10 14:32:20 ubuntu-vm systemd[1]: Starting User Manager for UID 1000...
```

Analysis of the above log entries:

1. 14:32:15 — Failed login attempt for user 'admin' from IP 192.168.1.100 — possible brute-force or scanning attack.
2. 14:32:18 — Connection dropped after the failed attempt (preauth stage).
3. 14:32:20 — Successful login for user 'student' from IP 192.168.1.50 — legitimate access.
4. 14:32:20 — PAM session opened for student (UID 1000), systemd created user slice and runtime directory — normal login process.

Common Errors & Fixes

Error / Problem	Solution
No journal files were found	Check /etc/systemd/journald.conf has Storage=persistent
Journal has been rotated since last boot	Use journalctl --list-boots to find the correct boot ID
Failed to get journal: Permission denied	sudo usermod -a -G systemd-journal student then logout/login
Logs show wrong time / timezone	Check: timedatectl status then set correct timezone
Journal consuming too much disk	sudo journalctl --vacuum-size=200M

Module 4: Practical Labs

VM Resource Warning

These labs are designed for VMs with 4–8 GB RAM. If your VirtualBox VM has less:

- Reduce stress test durations (change 60s to 10s)
- Close Firefox and other apps before starting
- If VM freezes, use VirtualBox Machine → Reset (data loss in open files)

Lab 1: Monitoring System Resources

Objective: Use monitoring tools to observe system behaviour under CPU load.

```
# Terminal 1: Start monitoring
top

# Terminal 2: Create CPU load (4 processes consuming CPU)
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &

# Observe in top (Terminal 1):
#   %CPU shows ~100% per 'yes' process (400% total on 4-core VM)
#   Load average increases to ~4.0
#   'yes' processes appear at the top of the list
#   us (user CPU) increases; id (idle) drops to near 0%

# Terminal 3: Monitor with vmstat
vmstat 1 30

# Observe in vmstat:
#   r (run queue) increases to 4
#   id (idle) drops to near 0
#   us (user CPU) increases to ~95%

# Stop load – in Terminal 2:
killall yes
```

Expected Results:

- Load average should spike to approximately the number of CPU cores
- CPU idle (id) should drop below 5%
- Run queue (r in vmstat) should show 4 — one per 'yes' process

Lab 2: Detecting Memory Pressure

```
# Terminal 1: Monitor memory
watch -n 1 free -h

# Terminal 2: Monitor VM stats
vmstat 1 30

# Terminal 3: Create memory pressure
sudo apt install stress
stress --vm 2 --vm-bytes 2G --timeout 60s

# Observe in Terminal 1 (free -h):
#   Available memory drops significantly
#   Swap usage may increase if RAM is exhausted

# Observe in Terminal 2 (vmstat):
#   si/so (swap in/out) should remain 0 if enough RAM available
#   If si/so > 0, the system is swapping – this is bad
#   free column decreases visibly

# Check for OOM killer events
journalctl -k | grep -i 'out of memory'

# stress auto-terminates after 60 seconds
```

Warning Signs:

- si or so > 0 in vmstat: System is swapping — add more RAM or reduce load
- swpd increases: Swap space is being used — investigate memory usage

Lab 3: I/O Performance Analysis

```
# Terminal 1: Monitor disk I/O
iostat -x 1 30

# Terminal 2: Create disk activity – writes 1 GB of zeros
dd if=/dev/zero of=/tmp/stress_test.bin bs=1M count=1024

# Watch in iostat (Terminal 1) for:
#   %util spikes toward 100%
#   w/s (writes per second) jumps up
#   kB/s (write throughput) shows MB/s
#   aqu-sz may increase (queue depth)
#   await increases – possibly > 20 ms

# Note: In VirtualBox, throughput may be lower than physical disk.
# This is normal – virtualisation adds overhead.

# Clean up
rm /tmp/stress_test.bin
```

Analysis Questions:

1. What was the peak %util? (Healthy < 70%, Critical > 90%)
2. What was the average await? (Healthy < 10 ms, Slow > 20 ms)
3. Did aqu-sz exceed 1? (If yes, requests were queuing — disk cannot keep up)

Lab 4: Tuning for Desktop Performance

```
# Step 1: Document settings BEFORE tuning
echo '=== BEFORE TUNING ==='
sysctl vm.swappiness
sysctl vm.vfs_cache_pressure

# Step 2: Apply temporary changes
echo '=== APPLYING TUNING ==='
sudo sysctl vm.swappiness=10
sudo sysctl vm.vfs_cache_pressure=50

# Step 3: Verify (AFTER)
echo '=== AFTER TUNING ==='
sysctl vm.swappiness
sysctl vm.vfs_cache_pressure

# Step 4: Make permanent
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-desktop.conf
echo 'vm.vfs_cache_pressure=50' | sudo tee -a /etc/sysctl.d/99-desktop.conf
sudo sysctl --system

# Step 5: Apply tuned profile
sudo apt install tuned
sudo tuned-adm profile desktop
sudo tuned-adm active

# Step 6: Verify tuned applied correctly
sudo tuned-adm verify
```

Parameter	Before (Default)	After	Effect
vm.swappiness	60	10	Less swapping; more responsive desktop
vm.vfs_cache_pressure	100	50	Cache retained longer; faster repeated file access

Lab 5: Log Analysis and Troubleshooting

```
# Scenario 1: Check for failed SSH login attempts
sudo grep 'Failed password' /var/log/auth.log | tail -10

# Scenario 2: Find system errors from today
journalctl -p err --since today

# Scenario 3: Track service startup
journalctl -u ssh --since '1 hour ago'
# (Install apache2 first if needed: sudo apt install apache2)

# Scenario 4: Export errors to a file for a report
journalctl --since '2026-04-10 00:00' --until '2026-04-10 23:59' -p err >
/tmp/today_errors.log
cat /tmp/today_errors.log

# Scenario 5: Monitor logs in real-time and generate a test entry
sudo tail -f /var/log/syslog &
logger 'Test message from student'
# Observe the message appearing in real-time
kill %1    # Stop the background tail

# Scenario 6: Investigate a specific time window
journalctl --since '30 minutes ago' --until '10 minutes ago' -p warning
```


Troubleshooting & Common Pitfalls

Golden Rule: Always take a VirtualBox snapshot BEFORE making major system changes!
 Use: Machine → Take Snapshot
 This lets you roll back instantly if something goes wrong.

System Monitoring Issues

Problem	Likely Cause	Quick Fix	Prevention
top shows 100% CPU but system is responsive	Multiple cores distributing load	Press 1 in top to see per-core usage	Understand load average vs CPU%
High load average but low CPU%	Processes waiting for disk	Run iostat -x 1; check %util and await	Add faster disk or reduce I/O
top shows 100% wa (I/O wait)	Disk bottleneck or VM host busy	Check host disk activity; reduce VM disk I/O	Use SSD for VM storage; enable Host I/O Cache
Zombie processes in top	Parent process not cleaning up children	Get parent: ps -o ppid= <zombie_pid> then restart parent service	Fix application code
htop colours look wrong	Terminal colour scheme conflict	Press F2 → Colors → choose different scheme	Use standard terminal emulator

Performance Tuning Issues

Problem	Likely Cause	Quick Fix	Prevention
System slow after sysctl changes	Wrong swappiness or dirty ratios	sudo sysctl --system then reboot and test	Change one parameter at a time
sysctl changes lost after reboot	Forgot to create config file	Add to /etc/sysctl.d/*.conf and run sudo sysctl --system	Always make changes persistent if they work
tuned-adm: profile not applied	Service not running	sudo systemctl restart tuned	Enable service on boot
Cannot install tuned	Universe repository not enabled	sudo add-apt-repository universe && sudo apt update	Check software sources during setup
System won't boot after tuning	Invalid kernel parameters in config	Boot recovery mode; remove bad file from /etc/sysctl.d/	Test all changes in VM snapshot first

Log Management Issues

Problem	Likely Cause	Quick Fix	Prevention
journalctl shows nothing	Logs rotated or not persistent	Check /etc/systemd/journald.conf: Storage=persistent	Configure retention settings
/var/log directory huge (several GB)	Log rotation not working	sudo logrotate -f /etc/logrotate.conf or delete old logs manually	Monitor disk usage weekly
Permission denied reading logs	Not in adm group	sudo usermod -a -G adm student then logout/login	Add user to adm group during setup
rsyslog not logging	Service stopped or crashed	sudo systemctl restart rsyslog	Monitor service status
Logs show wrong time/timezone	System timezone misconfigured	sudo timedatectl set-timezone Your/Timezone	Set timezone during OS installation

VirtualBox-Specific Issues

Problem	Likely Cause	Quick Fix
VM extremely slow; high steal time	Host overloaded with other VMs or apps	Close other VMs; give this VM more CPU cores in VirtualBox settings
Disk metrics seem too fast or unrealistic	VirtualBox disk abstraction and caching	Enable Host I/O Cache in VM settings → Storage tab
iostat shows 0 I/O during heavy activity	VM using cached I/O from host	Add --direct=1 flag to dd commands, or check host metrics instead
Memory shows less than allocated	Guest Additions not installed	Install VirtualBox Guest Additions (see setup section)
VM clock drifts over time	VM clock not syncing with host	Install Guest Additions or: sudo apt install ntp

Quick Reference Card

Monitoring Commands

Command	Purpose	Key Options	VM Note
top	Process monitoring	M=mem, P=CPU, k=kill, 1=cores, q=quit	May show VirtualBox overhead processes
htop	Enhanced process viewer	Interactive; mouse support; F-key menus	Install via apt first
iostat -x 1	Disk I/O statistics	-x extended, -h human-readable, 1=1-sec interval	Reflects virtual disk performance
vmstat 1	VM, CPU, I/O overview	-w wide output, -a active/inactive memory	Watch st (steal time) column
free -h	Memory usage summary	-h human-readable	Includes buffer/cache in output
df -h	Disk space usage	-h human-readable	Watch root (/) partition — can fill up
du -sh /path	Directory size	-s summary, -h human-readable	Use for /var/log monitoring
uptime	Load averages	Shows 1, 5, 15-min averages	Should be < number of CPU cores
mpstat -P ALL	Per-CPU core stats	-P ALL shows all cores	Check load distribution across cores
dmesg -T	Kernel ring buffer	-T human-readable timestamps	Check for hardware errors at boot

Tuning Commands

Command	Purpose	Example / Note
sysctl -a	List all kernel parameters	Pipe to less or grep to search
sysctl vm.swappiness	Read a parameter value	No sudo needed — safe to run
sudo sysctl vm.swappiness=10	Set a parameter temporarily	Resets on next reboot
sudo sysctl --system	Apply all config files	Reads all files in /etc/sysctl.d/
sudo tuned-adm list	List available profiles	Check which are installed
sudo tuned-adm profile desktop	Apply a tuning profile	May change system behaviour immediately
sudo tuned-adm active	Show currently active profile	Verify it was applied correctly
sudo tuned-adm verify	Verify settings are applied	Check for conflicts or overrides

Log Commands

Command	Purpose	Note
journalctl	All systemd journal logs	Use -b for current boot; -r for newest first
journalctl -u ssh -f	Follow a service log	Replace ssh with any unit name
journalctl -p err	Filter by priority level	err, warning, info, debug, emerg
journalctl --since '1 hour ago'	Filter by time	Supports natural language time expressions
journalctl --list-boots	List all recorded boots	Use boot number with -b flag
journalctl -o json-pretty	JSON output for scripting	All fields included in structured format
tail -f /var/log/syslog	Follow traditional text logs	Classic approach; still widely used
grep -i 'error' /var/log/syslog	Search text logs	Add -C 3 for surrounding context lines
logger 'Test message'	Write a message to syslog	Useful for testing log pipelines
journalctl --disk-usage	Show journal disk usage	Run before vacuuming to see current size

Keyboard Shortcuts – top

Key	Action
M	Sort by memory usage
P	Sort by CPU usage (default)
T	Sort by total CPU time consumed
k	Kill a process (enter PID when prompted)
r	Renice — change priority of a process
1	Show all CPU cores individually
c	Toggle full command path display
V	Forest view — show process tree
d	Change refresh delay in seconds
Space	Force immediate refresh
q	Quit top

Keyboard Shortcuts – htop

Key	Action
F1 or h	Help screen — all commands and colour key
F2 or S	Setup — colours, meters, columns
F3 or /	Search — find process by name
F4 or \	Filter — show only matching processes
F5 or t	Tree view — show parent-child relationships
F6 or >	Sort by selected column
F9 or k	Kill process — choose signal from menu
F10 or q	Quit htop

Safety Reminders

- Always take a VM snapshot before any sysctl or tuned changes.
- Never kill PID 1 (systemd) or processes with PID < 100 — these are system-critical.
- sudo gives full unrestricted system access — there are no 'are you sure?' prompts for dangerous commands.
- When in doubt about a command found online, learn and understand before running it.